
Task W7: Arcade High Score Tracker

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Scenario

You have been asked to help a small retro arcade turn a messy high score log into a clean, ranked report. The arcade keeps every score from every machine in one plain CSV file, in whatever order the scores happen to come in. Your program reads that file, ranks every entry from the highest score to the lowest, and writes a new CSV file with the finished report.

For this task, you will do all of the work from the Linux command line: creating your project folders, writing your Java class in a plain text editor, compiling it, and running it. You will not use Eclipse or any other IDE for this task. The point is to see, by hand, everything an IDE normally does for you behind the scenes.

This is an individual task. Complete it on your own, the same as any other Task or Lab in this course.

2 Learning Objectives

By the end of this task you will have practiced:

- laying out a Java project by hand from the command line, in the same **src**, **bin**, and **files** arrangement Eclipse has been building for you automatically all semester
- creating, editing, and saving a plain text file without an IDE
- compiling a Java source file with **javac** and running the compiled class with **java**, including the **-d** and **-cp** flags
- reading a CSV file line by line with **Scanner**, splitting each line into fields, and storing the results in an **ArrayList**
- sorting a list with a simple algorithm you already know, and writing formatted output to a new file with **PrintWriter**
- reading an existing program closely enough to make a small, deliberate change to it yourself

3 About Your Screenshots

Throughout this task, you will be asked to take a screenshot to show that you completed a step. A few things apply to all of them.

- Take one screenshot per marked step below. There are six in total, covering Sections 5 through 10.
- A screenshot does not need to show everything you did for that step. Get as much of the relevant terminal output on screen as you reasonably can.
- Each image must be an actual screenshot taken on your computer, not a photo taken with a phone or camera. If you do not already know how to take a screenshot on Xubuntu, look this up, for example a keyboard shortcut or a built in screenshot tool.
- **File naming.** Name each screenshot to match the section it documents: `taskw07_5.png` for Section 5, `taskw07_6.png` for Section 6, and so on through `taskw07_10.png`.

4 A Note on Editors

Everything in this task happens in a plain text editor and the terminal, not inside an IDE. Xubuntu already has `nano` and `vim` installed, so both are ready to use right away.

4.1 vim, the editor you are most likely to see me use

Open a file with vim by typing `vim FILENAME`. Vim starts in **Normal mode**, where the keys you press are commands rather than text. Press `i` to switch to **Insert mode**, where vim behaves like an ordinary editor and you can type normally. When you are done typing, press `Esc` to return to Normal mode.

From Normal mode, a few commands that begin with a colon save the file and quit.

```
1 :w      # save (write) the file
2 :q      # quit
3 :wq     # save and quit in one step
4 :q!     # quit without saving
```

If you ever feel stuck. Press `Esc` to make sure you are in Normal mode, then type `:q!` and press Enter. That leaves vim without saving, no matter what state you were in. Almost everyone needs this at least once, so it is worth remembering early.

4.2 nano, a simpler fallback

If you would rather start with something gentler than vim, `nano` is a friendlier editor. Open a file with `nano FILENAME`, type normally, then save with `Ctrl+O`, press Enter to confirm the file name, and exit with `Ctrl+X`.

4.3 GVim and NeoVim, two more options

You are not required to install or use either of these for this task, but you should know they exist.

GVim is a graphical version of vim. It opens in its own window with menus and mouse support, but the underlying editor, and every command shown above, is the same vim you already know. If you want to try it, install it with:

```
1 sudo apt install vim-gtk3
```

then open a file with `gvim FILENAME`.

NeoVim is a modern, actively maintained reimplement of vim. It runs in the terminal the same way vim does, keeps the same modes and most of the same commands, and adds a more current plugin system and configuration language underneath. If you want to try it, install it with:

```
1 sudo apt install neovim
```

then open a file with `nvim FILENAME`.

Table 1: A quick comparison of the four editors mentioned in this task.

Editor	Interface	Notes
vim	Terminal only	Installed already. The editor you are most likely to see me use.
nano	Terminal only	Installed already. The simplest option to start with.
GVim	Graphical window	Same vim underneath, with menus and mouse support. Optional install.
NeoVim	Terminal, with optional graphical front ends	A modern, plugin friendly reimplement of vim. Optional install.

One more simple option. Xubuntu also includes **Mousepad**, a basic graphical text editor, if you would rather work outside the terminal. Open it from the Applications menu, or type `mousepad FILENAME` from inside your project folder. Mousepad uses its own save dialog, so pay close attention to where each file is saved. If you open it from the Applications menu, the dialog may default to your home folder or Desktop. Make sure every file ends up in the correct place inside your project folder, for example `files/scores.csv` or `src/arcade/ArcadeScores.java`.

4.4 These editors in the wider industry

Vim and its descendants are not just classroom tools. Many professional developers, particularly in backend, systems, and open source work, use vim or NeoVim as their everyday editor, often alongside a terminal multiplexer such as tmux. A few videos, found with web search, give a sense of how this looks in practice.

- [Fireship \(YouTube channel\)](#), a widely watched programming channel whose short "100 Seconds of Code" series includes quick explainers on both vim and NeoVim.
- [ThePrimeagen \(YouTube channel\)](#), run by a former Netflix engineer who now codes and streams almost entirely in NeoVim, with videos that walk through a real, terminal based development setup.
- [Lex Fridman Podcast #461: ThePrimeagen](#), a long form conversation that includes a section on why he uses NeoVim and how a terminal based workflow fits into a professional engineering career.

A note on these resources

These links were found with help from an AI system and web search, not from the instructor's own prior use. They look relevant, but none have been personally verified by the instructor. Treat them as a starting point, and let your instructor know if one is broken, off-topic, or wrong.

5 Project Setup

A note on your environment. Your Xubuntu VM already includes Firefox, so you can open this assignment, Moodle, and GitHub entirely from inside the VM without switching back to your host machine. You may complete this task on a different Linux machine instead of the VM if you prefer, but you are responsible for installing and configuring everything yourself (a JDK, git, and so on). If you need a refresher on using the VM, see Lab W1.

Open a terminal (**Applications > Terminal Emulator**, or **Ctrl+Alt+T**) and build the following folder structure by hand. This mirrors the **src**, **bin**, and **files** layout Eclipse has been building for you automatically all semester: **src** for your Java source files, **bin** for the compiled class files, and **files** for anything your program reads or writes.

1. Create the project folders.

```
1 mkdir -p ~/cis3300/taskw07/src/arcade
2 mkdir -p ~/cis3300/taskw07/bin
3 mkdir -p ~/cis3300/taskw07/files
```

2. Move into the project folder. You will run every command for the rest of this task from here.

```
1 cd ~/cis3300/taskw07
```

3. Confirm the structure.

```
1 ls -R
```

Take a screenshot of the `ls -R` output showing the **src**, **bin**, and **files** folders, and save it as `taskw07_5.png`.

6 Creating the Data File

The arcade's raw score log is a simple CSV file: a header row, then one row per score, with the player's name, the game title, and the score as an integer.

Open a new file at `files/scores.csv` in the editor of your choice (vim, nano, GVim, NeoVim, or Mousepad) and type the following exactly as written, including the header row.

```
1 name,game,score
2 Maria,Galaxy Invaders,15200
3 Deshawn,Retro Racer,9800
4 Priya,Galaxy Invaders,21300
5 Wyatt,Pixel Dungeon,7600
6 Alexis,Retro Racer,12500
7 Noah,Pixel Dungeon,15200
```

```
8 Grace,Galaxy Invaders,18400
9 Malik,Retro Racer,9800
10 Sofia,Pixel Dungeon,20100
11 Ethan,Galaxy Invaders,11000
```

It is fine to copy and paste this data instead of typing it, but check the result carefully afterward. A paste from a PDF does not always come through cleanly, and extra spaces can sneak in around the commas. There should be no space before or after any comma in the file.

Save the file, then confirm its contents from the command line.

```
1 cat files/scores.csv
```

Take a screenshot of the `cat` output and save it as `taskw07_6.png`.

7 Creating the Java Source File

Below is the full program you will build. Read through it once before you start typing, so you know roughly what each part is doing.

`ArcadeScores` does three things, in order: it reads every row of `files/scores.csv` into a list, it sorts that list from the highest score to the lowest using a simple selection sort, and it writes a new file, `files/ranked_scores.csv`, with a rank number added to every row.

7.1 Reading the file

The `readScores` method opens `files/scores.csv` with a `Scanner`, reads and discards the header row, and then reads every remaining line. Each line is split on its commas into a `String[]` with three elements (name, game, score), and every one of those arrays is added to an `ArrayList`.

7.2 Sorting the list

The `sortByScoreDescending` method is a selection sort. For each position in the list, it scans the rest of the list for the highest remaining score, then swaps that row into the current position. This is the same idea as sorting a hand of cards by repeatedly picking the best card left and moving it to the front.

7.3 Writing the report

The `writeRanking` method opens `files/ranked_scores.csv` with a `PrintWriter`, writes a header row, and then writes one line per row in the now sorted list, numbering them starting at 1.

Create the file `src/arcade/ArcadeScores.java` in your editor and type the following exactly as written.

```
1 package arcade;
2
3 import java.io.File;
4 import java.io.FileNotFoundException;
5 import java.io.FileWriter;
6 import java.io.IOException;
7 import java.io.PrintWriter;
8 import java.util.ArrayList;
9 import java.util.List;
10 import java.util.Scanner;
11
```

```

12  /*
13  * ArcadeScores reads a raw high score log from files/scores.csv and
14  * writes a ranked report to files/ranked_scores.csv. Each input row
15  * has three fields: a player name, a game title, and an integer
16  * score. The output adds a rank column, with the highest score
17  * ranked first.
18  */
19  public class ArcadeScores {
20
21      public static void main(String[] args) throws IOException {
22
23          String inputPath = "files/scores.csv";
24          String outputPath = "files/ranked_scores.csv";
25
26          List<String[]> rows = readScores(inputPath);
27          sortByScoreDescending(rows);
28          writeRanking(rows, outputPath);
29
30          System.out.println("Wrote " + rows.size() + " ranked scores to " +
              outputPath);
31      }
32
33      // Reads scores.csv, skips the header row, and returns each data
34      // row as a String array: {name, game, score}.
35      private static List<String[]> readScores(String path) throws
          FileNotFoundException {
36          List<String[]> rows = new ArrayList<>();
37          Scanner fileScanner = new Scanner(new File(path));
38
39          if (fileScanner.hasNextLine()) {
40              fileScanner.nextLine(); // skip the header row
41          }
42
43          while (fileScanner.hasNextLine()) {
44              String line = fileScanner.nextLine();
45              if (line.isBlank()) {
46                  continue;
47              }
48              rows.add(line.split(","));
49          }
50
51          fileScanner.close();
52          return rows;
53      }
54
55      // Sorts rows by score, highest first, using a simple selection
56      // sort. On each pass, it finds the highest remaining score and
57      // swaps it into place.
58      private static void sortByScoreDescending(List<String[]> rows) {
59          for (int i = 0; i < rows.size() - 1; i++) {
60              int maxIndex = i;
61              for (int j = i + 1; j < rows.size(); j++) {
62                  int scoreJ = Integer.parseInt(rows.get(j)[2]);
63                  int scoreMax = Integer.parseInt(rows.get(maxIndex)[2]);

```

```

64         if (scoreJ > scoreMax) {
65             maxIndex = j;
66         }
67     }
68     String[] temp = rows.get(i);
69     rows.set(i, rows.get(maxIndex));
70     rows.set(maxIndex, temp);
71 }
72 }
73
74 // Writes the ranked rows to a new CSV file, numbering them
75 // starting at 1 in the order they are given.
76 private static void writeRanking(List<String[]> rows, String path)
77     throws IOException {
78     PrintWriter writer = new PrintWriter(new FileWriter(path));
79     writer.println("rank,name,game,score");
80
81     int rank = 1;
82     for (String[] row : rows) {
83         String name = row[0];
84         String game = row[1];
85         String score = row[2];
86         writer.println(rank + "," + name + "," + game + "," + score);
87         rank++;
88     }
89     writer.close();
90 }
91 }

```

It is fine to copy and paste this code instead of typing it, but you will likely need to fix the line spacing and tabs afterward, since a paste from a PDF does not preserve indentation cleanly. **Formatting matters.** Code that is not formatted nicely will not receive credit.

Save the file when you are done typing.

Take a screenshot of your editor with `ArcadeScores.java` open, showing at least part of the code on screen, and save it as `taskw07_7.png`.

8 Compiling and Running

The `javac` command compiles your source file into bytecode, and the `java` command runs it. Because your class lives in the `arcade` package, both commands need a little more than the bare file name.

1. Compile the source file into the `bin` folder.

```
1 javac -d bin src/arcade/ArcadeScores.java
```

The `-d bin` flag tells `javac` where to put the compiled `.class` file. Because the class declares `package arcade`;, `javac` creates an `arcade` folder inside `bin` for you.

2. Confirm the class file was created.

```
1 ls bin/arcade
```

You should see `ArcadeScores.class`. This is the compiled bytecode; it is what `java` actually runs, not the `.java` source file.

3. Run the compiled class.

```
1 java -cp bin arcade.ArcadeScores
```

The `-cp bin` flag tells `java` where to look for compiled classes, and `arcade.ArcadeScores` is the fully qualified class name (package name, a dot, then the class name), not a file name.

If everything worked, you will see a line similar to this.

```
1 Wrote 10 ranked scores to files/ranked_scores.csv
```

Take one screenshot showing the compile command, the `ls bin/arcade` output, the run command, and its printed output together in one screen, and save it as `taskw07_8.png`.

9 Checking the Output

Look at the file your program wrote.

```
1 cat files/ranked_scores.csv
```

If your program worked correctly and you typed the data file exactly as given, your output should match the following exactly.

```
1 rank,name,game,score
2 1,Priya,Galaxy Invaders,21300
3 2,Sofia,Pixel Dungeon,20100
4 3,Grace,Galaxy Invaders,18400
5 4,Noah,Pixel Dungeon,15200
6 5,Maria,Galaxy Invaders,15200
7 6,Alexis,Retro Racer,12500
8 7,Ethan,Galaxy Invaders,11000
9 8,Malik,Retro Racer,9800
10 9,Deshawn,Retro Racer,9800
11 10,Wyatt,Pixel Dungeon,7600
```

If your file does not match, do not move on yet. Compare your `scores.csv` against Section 6 character for character, and compare your `ArcadeScores.java` against Section 7 the same way. A single missing comma or a mistyped variable name is the most common cause.

Take a screenshot of your own `cat` output and save it as `taskw07_9.png`.

10 The Challenge: Add a Medal Column

The arcade owner has one more request. The ranked report should show a medal for the top three scores: **Gold** for rank 1, **Silver** for rank 2, and **Bronze** for rank 3. Every other row should show a dash, `-`, in that column instead.

The finished output should look like this, with your own column added.

```
1 rank,name,game,score,medal
2 1,Priya,Galaxy Invaders,21300,Gold
3 2,Sofia,Pixel Dungeon,20100,Silver
4 3,Grace,Galaxy Invaders,18400,Bronze
5 4,Noah,Pixel Dungeon,15200,-
```

You are not given the code for this part.

- Only the part of the program that writes the output needs to change. You do not need to touch how the file is read or how it is sorted.
- The header row needs a new column name.
- Inside the loop that writes each row, you already have the current **rank** as a number. Decide what that row's medal should be based on that number, and add it to the line you write.

Edit `src/arcade/ArcadeScores.java` in your editor, save it, then recompile and rerun it the same way you did in Section 8.

```
1 javac -d bin src/arcade/ArcadeScores.java
2 java -cp bin arcade.ArcadeScores
```

Check the result.

```
1 cat files/ranked_scores.csv
```

Take one screenshot showing your recompile, rerun, and the final `cat` output together in one screen, and save it as `taskw07_10.png`.

11 Grading

This task is worth 29 points. Each item below is either met or not met; there is no partial credit within an item. Each table lists one category of items with its point value.

Formatting. Severely poor formatting may result in a zero for the entire task.

Table 2: Grading checklist, Submission and Setup (1 point each, 4 points).

Met	Item	Points
☐	<code>taskw07</code> folder is present in the <code>cis3300</code> repository, following the structure shown in Section 5	1
☐	<code>ArcadeScores.java</code> compiles without errors from the command line	1
☐	<code>ArcadeScores.java</code> runs without errors from the command line	1
☐	<code>bin</code> folder is not included in the submitted repository	1

Table 3: Grading checklist, Screenshots (1 point each, 6 points).

Met	Item	Points
☐	<code>taskw07_5.png</code> is present, showing the <code>ls -R</code> output from Section 5	1
☐	<code>taskw07_6.png</code> is present, showing the <code>cat</code> output from Section 6	1
☐	<code>taskw07_7.png</code> is present, showing the editor open with <code>ArcadeScores.java</code> from Section 7	1
☐	<code>taskw07_8.png</code> is present, showing the compile and run output from Section 8	1
☐	<code>taskw07_9.png</code> is present, showing the <code>cat</code> output from Section 9	1
☐	<code>taskw07_10.png</code> is present, showing the recompile, rerun, and final <code>cat</code> output from Section 10	1

Table 4: Grading checklist, Base Program Correctness (2 points each, 10 points).

Met	Item	Points
☐	<code>files/scores.csv</code> contains the data exactly as given, unmodified	2
☐	<code>readScores</code> correctly reads all ten rows, skipping the header row	2
☐	<code>sortByScoreDescending</code> correctly sorts the list from highest score to lowest	2
☐	<code>writeRanking</code> writes <code>files/ranked_scores.csv</code> with a header row and rank numbers starting at 1	2
☐	Output in <code>files/ranked_scores.csv</code> matched the Expected Output in Section 9 before the challenge change	2

Table 5: Grading checklist, The Challenge: Medal Column (2 points each, 4 points).

Met	Item	Points
☐	Header row includes a medal column, and the reading and sorting logic is unchanged from the base program	2
☐	Every row shows the correct medal (Gold , Silver , Bronze , or a dash) for its rank	2

Table 6: Grading checklist, Code Quality (1 point each, 5 points).

Met	Item	Points
☐	Indentation is consistent throughout <code>ArcadeScores.java</code>	1
☐	Code is well formatted and follows standard Java coding conventions	1
☐	The header comment block is still present at the top of the file	1
☐	No leftover debug output or commented-out code beyond what was provided	1
☐	No unrequested extra code or output added beyond what the assignment asked for	1

12 Submission

Remember that your Xubuntu VM includes Firefox, so you can submit your work directly from the VM without moving files to your host machine first.

1. Inside your `cis3300` repository, you should have a `taskw07` folder containing your `src` and `files` folders, and your six screenshots. Do not include the `bin` folder; compiled class files do not belong in your repository.
2. Add the `taskw07` folder to your repository, using either option:
 - commit and push from the terminal with git, or
 - upload the folder through the GitHub website in Firefox (leave out the `bin` folder).
3. Return to Task W7 in Moodle and type **done** in the text submission box. Do not upload images to the submission box.